

FitBuddy

AI Fitness Plan Generation Using Gemini Models

PROJECT DOCUMENTATION

Prepared in a milestone-based format inspired by the supplied sample documentation.

Project title: FitBuddy – AI Fitness Plan Generation Using Gemini Models

Technology details and implementation examples in this document are a proposed baseline; adapt them to the final application.

Project Description

FitBuddy is an AI-assisted fitness planning application designed to help users create structured workout plans based on their fitness goals, experience level, available equipment, preferred workout duration, and weekly availability. By integrating a Gemini model through the Google AI developer interface, the application can turn user preferences into an organized plan that is easier to understand and follow.

The proposed system includes a Fitness Plan Generator that creates a weekly workout schedule, a Workout Detail section that describes exercises, sets, repetitions or duration, rest periods, and a Progress Guidance section with practical consistency tips. Depending on the final scope, the system may also provide warm-up and cool-down suggestions, exercise substitutions, and a simple plan summary users can save or copy.

FitBuddy is intended for general fitness education and planning. It does not diagnose medical conditions or replace a qualified healthcare professional or certified trainer. Generated plans should include a safety note and encourage users to stop any exercise that causes pain or unusual symptoms.

Core Modules

- User Profile and Goal Input: collects fitness goal, experience level, weekly availability, session duration, equipment access, and optional preferences.
- AI Fitness Plan Generator: sends a structured prompt to a Gemini model and receives a plan tailored to the submitted preferences.
- Workout Schedule Renderer: presents the generated plan by day, with exercise name, sets/repetitions or duration, rest guidance, and notes.
- Plan Review and Regeneration: allows users to revise their inputs and request a new plan.
- Safety and Input Validation: validates required fields, constrains input lengths, and displays a reminder that AI-generated plans require personal judgment.

Scenarios

Scenario 1: Beginner Seeking General Fitness

A beginner selects general fitness, chooses three workout days per week, indicates that no gym equipment is available, and sets a 25-minute session duration. FitBuddy generates a simple bodyweight schedule with beginner-friendly movements, rest days, and brief exercise instructions.

Scenario 2: Strength-Focused User

A user with some training experience selects strength development, reports access to dumbbells, and chooses four weekly sessions. FitBuddy creates a structured split that distributes muscle groups across the week and includes set and repetition guidance, rest intervals, and options for adapting the plan.

Scenario 3: Limited Time and Equipment

A user who can train for only 15–20 minutes at home selects a suitable goal and schedule. FitBuddy proposes short sessions using available equipment and includes a reminder to prioritize safe technique over speed.

Scope and Assumptions

- This document assumes a web application with a Python Flask backend and an HTML/CSS/JavaScript frontend. Replace these choices if the actual project uses a different stack.
- Gemini model access requires a valid API key and is subject to provider availability, quotas, and model changes.
- The initial version does not require a database. If accounts, saved plans, or progress history are added, include an ER diagram and database schema.
- Generated exercise suggestions are informational and should be reviewed for suitability before use.

Technical Architecture

FitBuddy can use a three-tier architecture: a browser-based frontend, a Flask application backend, and the Gemini API for plan generation. The frontend gathers preferences and displays results. The backend validates and normalizes inputs, builds a structured prompt, calls the model, checks the returned content, and sends a predictable response to the browser.

Architecture Components

- Frontend: HTML forms, CSS layout, and JavaScript for asynchronous submission and dynamic results.
- Backend: Flask routes for the home page and plan-generation requests; validation, prompt construction, error handling, and response formatting.
- AI integration: Google Gemini API client, prompt template, generation configuration, and response parsing.
- Configuration: environment variables stored outside source control, including the Gemini API key.

Request Flow

1. The user opens the FitBuddy generator page.
2. The user enters fitness goals, experience, weekly frequency, session duration, and equipment availability.
3. The frontend sends the form data as JSON to the backend generation endpoint.
4. The backend validates fields and rejects missing or unreasonable values.
5. The backend constructs a prompt that asks Gemini for a structured, safety-conscious fitness plan.
6. The Gemini API returns generated content; the backend parses and checks required sections.
7. The backend returns the plan as JSON, and the frontend renders the schedule and safety note.

Illustrative Architecture Diagram



The diagram is a text illustration. For a final submission, it may be replaced with a drawn block diagram showing request and response directions.

Pre-requisites

- Python 3.10 or a version supported by the selected dependencies.
- Basic knowledge of Python, Flask, HTML, CSS, and JavaScript.
- A Google AI Studio / Gemini API key and familiarity with the current Gemini API documentation.
- pip and a virtual environment for dependency isolation.
- A code editor and a browser for development and testing.
- Optional: Git for version control and a hosting service for deployment.

Project Workflow

Milestone 1: Requirements, Model Selection, and Architecture

- Activity 1.1: Identify the required fitness goals, profile fields, output format, and safety constraints.
- Activity 1.2: Review the available Gemini models and select a model compatible with the current API and project requirements.
- Activity 1.3: Define the frontend, backend, and AI integration architecture.
- Activity 1.4: Create the development environment and configure the API key securely.

Milestone 2: Core Functionality Development

- Activity 2.1: Build the fitness profile form and define supported values for experience, goal, schedule, duration, and equipment.
- Activity 2.2: Implement prompt construction and Gemini API communication.
- Activity 2.3: Define the expected output structure for weekly plans and exercise details.

Milestone 3: Flask Backend Development

- Activity 3.1: Create routes for the home page and fitness-plan generation.
- Activity 3.2: Add validation, API error handling, response parsing, and required-field checks.
- Activity 3.3: Return a consistent JSON response to the frontend.

Milestone 4: Frontend Development

- Activity 4.1: Design a responsive interface for profile entry and plan viewing.
- Activity 4.2: Display the generated plan by day and include loading, success, and error states.
- Activity 4.3: Add copy, regenerate, and reset actions if included in the final scope.

Milestone 5: Testing and Deployment

- Activity 5.1: Test form validation, API integration, output parsing, and responsive behavior.
- Activity 5.2: Run the app locally and verify it with several representative input combinations.
- Activity 5.3: Deploy using a chosen hosting platform and configure secrets through the platform's environment settings.

Milestone 6: Conclusion and Future Enhancements

Review the delivered features, document limitations, and identify future enhancements such as user accounts, saved plans, progress tracking, exercise illustrations, multilingual output, and integration with wearable data.

Milestone 1: Requirements and Model Setup

Activity 1.1: Define User Inputs

The application should collect only the information needed to generate a useful plan. A proposed input set is listed below.

- Fitness goal: general fitness, strength, endurance, flexibility, or another supported goal.
- Experience level: beginner, intermediate, or advanced.
- Weekly training days: a bounded number of days chosen by the user.
- Session duration: a realistic number of minutes per session.
- Equipment: bodyweight only, dumbbells, resistance bands, gym equipment, or user-specified options.
- Preferences and constraints: optional text for exercise preferences, with a warning not to enter sensitive medical details.

Activity 1.2: Configure Gemini Access

8. Open Google AI Studio and sign in with an eligible Google account.
9. Create or select a project/API key according to the provider's current setup instructions.
10. Store the key in an environment variable or a local .env file that is excluded from version control.
11. Install the current supported Google Gen AI Python SDK and verify the installed package against its official documentation.
12. Choose a Gemini model that is currently available to the account and appropriate for structured text generation.

Important: model names, SDK methods, and usage limits can change. Confirm the current official Gemini documentation before final implementation rather than relying on a hard-coded model name from an old tutorial.

Activity 1.3: Environment Setup

```
python-mvenv.venv
# Windows
.venv\Scripts\activate
# macOS/Linux
source .venv/bin/activate
pipinstallflask python-dotenvgoogle-genai
```

The dependency list above is a proposed starting point. Pin tested versions in requirements.txt before submitting or deploying the project.

Activity 1.4: Project Structure

```
fitbuddy/
├── app.py
├── requirements.txt
├── .env          #localsecret; do not commit
├── .gitignore
├── templates/
│   └── index.html
├── static/
│   ├── css/
│   │   └── style.css
│   └── js/
│       └── main.js
```

Milestone 2: Core Functionality Development

Activity 2.1: Fitness Plan Generator

The plan generator combines the user's profile with a carefully written prompt. The prompt should ask for a weekly schedule that matches the requested number of days and session duration, explains exercises clearly, includes warm-up and cool-down suggestions, and provides reasonable rest periods. The model should avoid presenting uncertain recommendations as medical advice.

Suggested Output Structure

```
{
  "success": true,
  "plan_title": "Example weekly plan",
  "goal": "general fitness",
  "weekly_schedule": [
    {
      "day": "Day 1",
      "focus": "Full body",
      "duration_minutes": 25,
      "exercises": [
        {
          "name": "Example movement",
          "sets": 2,
          "reps": "8-10",
          "rest_seconds": 60,
          "notes": "Use controlled, comfortable movement."
        }
      ]
    }
  ],
  "general_tips": ["Start gradually and prioritize technique."],
  "safety_note": "Adapt or stop exercises that cause pain."
}
```

This is an illustrative schema, not a guaranteed model response. The backend should validate that the returned object contains the expected fields and handle malformed output safely.

Activity 2.2: Prompt Design

A useful prompt includes the user inputs, output schema, tone, and safety constraints. It should tell the model not to invent medical diagnoses, to avoid extreme training volumes, and to ask for clarification or provide conservative general guidance when required information is insufficient.

SYSTEMROLE (illustrative):

"You are a fitness-planning assistant. Create general, beginner-conscious exercise plans. Return structured JSON matching the requested schema. Respect the user's stated time, equipment, and experience. Include warm-up, rest, and a safety note. Do not diagnose conditions or claim that the plan is medically appropriate for every person."

Activity 2.3: Input Validation

- Require a supported goal and experience level.

- Check that training days and duration are within configured application limits.
- Limit the length of free-text preferences.
- Treat all user input as untrusted; do not execute generated text as code or HTML.
- Return a clear client error for invalid fields and a generic server error for API failures.

Milestone 3: Flask Backend Development

Activity 3.1: Define Application Routes

A minimal Flask implementation may expose the home page and one JSON endpoint. The following is a structural example; adapt it to the exact SDK version and response format selected for the project.

```
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json(silent=True) or {}
    goal = str(data.get("goal", "")).strip()
    level = str(data.get("level", "")).strip()
    # Validate all fields before calling Gemini.
    if not goal or not level:
        return jsonify({"success": False, "error": "Goal and level are required."}), 400
    # Build prompt, call Gemini, parse and validate the model response here.
    # Return a structured plan only after checking the response.
    return jsonify({"success": False, "error": "Generation is not configured yet."}), 501
```

The example deliberately leaves the provider call as a marked implementation step. This avoids suggesting that an untested SDK invocation is ready for production.

Activity 3.2: Gemini API Integration

- Load configuration from environment variables at application startup.
- Initialize the current supported Google Gen AI client using the secret API key.
- Build a prompt from validated input rather than concatenating unchecked data into a system instruction.
- Call the selected Gemini model with suitable generation settings and a bounded output length.
- Parse the response and validate its structure before returning it to the browser.
- Handle rate limits, network failures, safety blocks, and unexpected response formats without exposing secrets or stack traces.

Activity 3.3: Error Handling

- 400 Bad Request: required fields are missing or values are outside supported limits.
- 429 or provider rate-limit response: show a retry-later message without repeated automatic calls.
- 502/500: generation failed or the returned content could not be validated.
- Never return the API key, raw environment configuration, or internal traceback to the client.

Security Considerations

- Keep .env out of source control and rotate a key if it is exposed.
- Do not embed the Gemini API key in client-side JavaScript.
- Use HTTPS in production and configure host-level secrets.
- Apply request-size limits and consider rate limiting to reduce abuse and unexpected API costs.

Milestone 4: Frontend Development

Activity 4.1: Build the Input Form

The main page can use a clean fitness dashboard design with a header, brief explanation, form card, and results area. Suggested controls include goal selection, experience level, training days, workout duration, equipment, and an optional preferences field.

Activity 4.2: Render Generated Results

- Display a plan title and a summary of the selected goal and preferences.
- Organize workout sessions by day, with exercise name, sets/repetitions or duration, rest, and notes.
- Separate general tips from the safety note so users can find them easily.
- Show a loading indicator while the request is processing.
- Display validation and network errors in accessible, understandable language.
- Render model-generated content as text, not trusted HTML, to reduce injection risks.

Activity 4.3: Suggested Page Layout

[FitBuddy header and tagline]
[Fitness goal | Experience level | Days per week]
[Session duration | Equipment | Optional preferences]
[Generate Fitness Plan button]
[Loading / error message]
[Weekly plan: Day cards and exercise details]
[General tips | Safety note | Regenerate / Reset]

For the final report, insert actual screenshots of the implemented website in this section. Do not label mockups or illustrative layouts as screenshots of a working application.

Milestone 5: Testing and Deployment

Activity 5.1: Functional Testing

Test Case	Input / Condition	Expected Result
Valid input	A supported goal and profile are submitted.	A structured plan is displayed.
Missing required field	Goal or experience is empty.	A clear validation message appears; no model call is made.
Out-of-range duration	Duration is outside configured limits.	The backend rejects the request.
API key missing	The environment variable is absent.	The application reports a configuration error without exposing secrets.
Provider failure	The API times out or returns an error.	A user-friendly error appears and the interface remains usable.
Malformed model output	The response does not match the schema.	The backend rejects it safely and reports generation failure.
Responsive layout	The page is opened on mobile and desktop sizes.	Inputs and results remain readable and usable.
Safety wording	A generated plan is displayed.	A visible general safety note is included.

Activity 5.2: Run Locally

```
python-mvenv.venv
# Activate the environment, then:
pipinstall-r requirements.txt
pythonapp.py
```

Open the local address printed by Flask in the terminal. Confirm the configured port and debug setting in app.py. Debug mode should not be enabled on a public production deployment.

Activity 5.3: Deployment

Deploy FitBuddy to a hosting provider that supports the selected Python runtime. Configure the Gemini API key using the host's secret/environment-variable settings, install dependencies from requirements.txt, and set a production WSGI server and HTTPS. A temporary tunnelling tool may be used for demonstrations, but it should not be treated as a substitute for production hosting.

- Verify the deployed health and home routes.
- Test one valid request and representative validation/error cases.
- Review API quotas and set usage controls where available.
- Do not publish .env, API keys, debug logs, or user data.

Exploring the Website's Web Pages

Home / Fitness Planner Page

Description: The home page introduces FitBuddy and provides the entry point for generating a fitness plan. It contains the project name, a short explanation of the tool, and the fitness profile form.

Input Section

Description: The input section allows the user to choose a fitness goal and experience level, specify available training days and session duration, and select equipment. Optional preferences may be provided as short free text. A Generate Fitness Plan button starts the request.

Generated Plan Section

Description: The results section displays a weekly schedule in day-based cards. Each session lists the focus and planned exercises, with sets, repetitions or duration, rest guidance, and notes where available. General tips and a safety note are shown separately.

Error and Loading States

Description: While the request is running, the interface indicates that generation is in progress and prevents accidental duplicate submissions. If validation or API errors occur, the interface displays a clear message and lets the user revise the form and try again.

Screenshot Placeholders for Final Submission

- Figure 1: FitBuddy home page (insert actual screenshot).
- Figure 2: Fitness preferences form (insert actual screenshot).
- Figure 3: Generated weekly fitness plan (insert actual screenshot).
- Figure 4: Validation or loading state (optional; insert actual screenshot).

Conclusion

FitBuddy – AI Fitness Plan Generation Using Gemini Models is a proposed AI-assisted application for generating structured workout plans from user preferences. The project brings together a browser-based interface, backend input validation, prompt engineering, Gemini model integration, and dynamic presentation of generated results.

The milestone-based approach supports incremental development: first define requirements and model access, then implement plan generation, connect the backend, build the interface, and test deployment. Particular attention should be given to secure API-key management, output validation, accessible design, and transparent safety messaging.

Future enhancements may include user accounts, saving and comparing plans, progress tracking, exercise demonstrations, plan export, multilingual support, and carefully reviewed integrations with fitness devices. Any additional health-related features should be designed with appropriate privacy, safety, and professional-review considerations.

References and Documentation to Consult

- Google AI for Developers — Gemini API documentation: <https://ai.google.dev/gemini-api/docs>
- Flask documentation: <https://flask.palletsprojects.com/>
- Python documentation: <https://docs.python.org/3/>
- Python dotenv package documentation: <https://pypi.org/project/python-dotenv/>
- HTML, CSS, and JavaScript references: <https://developer.mozilla.org/>

Note: URLs are included as starting points. Check current provider documentation for supported model identifiers, SDK methods, and deployment requirements before implementing the application.

Project Configuration Checklist

- Confirm final technology stack and project folder structure.
- Select a currently supported Gemini model and verify its access requirements.
- Create the local environment file and ensure it is excluded from Git.
- Implement the actual Gemini API call and structured response parsing.
- Set and document validation limits for each form field.
- Test success, invalid input, provider errors, and malformed output.
- Capture screenshots from the working application and replace the placeholders in this document.
- Confirm whether the project uses a database. If yes, add ER diagram, schema, and database description.
- Update package versions, run commands, and deployment instructions to match the completed implementation.